Laborator 8 - Integrare

Ce înseamnă "integrare"? (5 min)

Cu puține excepții, proiectele software nu sunt izolate și auto-suficiente, ci depind de alte sisteme, fiind părți dintr-un întreg mai mare. Utilizatorii se așteaptă să poată folosi transparent toate funcțiile acestui întreg, indiferent de componenta software din care o anumită funcție face parte. Pentru aceasta, diferitele componente trebuie conectate impreună. **Acest proces este integrarea**. De obicei, ea constă în "cablarea" datelor și informației între componente.

Această "cablare" presupune de obicei:

- definirea datelor de care diferitele componente au nevoie
- definirea transferului acestor date (sursă, storage comun, destinație)
- definirea interfețelor prin care componentele să comunice între ele
- definirea hardware-ului pe care o componentă software va fi instalată ("deployed")
- în cazul în care componenta în cauză are ea însăși subcomponente conectate, pașii de mai sus trebuie repetați pentru subcomponente *(ex. un același proiect software poate avea nevoie de mai multe tipuri de hardware: serverul fizic de baze de date, servere, middle tier - fiecare având instalate doar anumite module și nu tot proiectul)*

Integrarea: un task dificil?

De obicei, **DA**. Motivele sunt mai multe:

Cerințe divergente

Cerințele integrării sunt de multe ori în contradicție cu cerințele de arhitectură ale proiectului. *Ex. din motive de performanță, o componentă software ar putea avea nevoie de un storage local mai curând decât de date remote. Integrarea acestei componente într-un sistem ar putea însă presupune accesul remote la acele date, printr-o interfață.*

Evoluția sistemului

Un sistem evoluează în timp: se adaugă noi funcționalități, noi componente, noi date. De multe ori, sistemul nu mai este optimal pentru cerințele noii componente - sau componentele mai vechi se dovedesc depășite de noile cerințe.

Complexitatea

Un sistem are adeseori o complexitate imposibil de stăpânit de către un singur om. Integrarea poate fi deci un proiect în sine, necesitând conlucrarea unei întregi echipe.

Impactul

Integrarea unei componente noi poate afecta funcționarea celorlalte. *De exemplu: o componentă nouă X accesează o componentă preexistentă, Y. Componenta X poate transfera date eronate care să provoace inconsistențe la nivelul datelor din Y, sau să scoată la iveală buguri preexistente (și anterior necunoscute) din componenta Y.*

Securitatea

De asemenea, mai ales în mediul de Web, pot apărea *breșe de securitate*. În funcție de modul de cuplare al componentelor, aceste breșe de securitate pot fi mai mult sau mai puțin periculoase. *De exemplu: accesul*

direct a unei componente de pe server la o bază de date poate introduce o breșă în securitatea bazei de date. Compromiterea serverului atrage după sine și compromiterea datelor din baza de date.

Abordări de Integrare

Integrarea stea

Integrarea "stea" (sau "spaghetti") presupune interconectarea fiecărei componente cu toate celelalte, refolosind la maximum funcționalitățile existente.

Avantajul este economia: refolosirea la maximum a funcționalităților existente în fiecare sistem duce la simplificarea componentelor, care trebuie doar să implementeze interfețe și nu să duplice funcționalități.

Dezavantajul rezidă însă la nivelul evoluției sistemului: pe masură ce crește numărul de componente, crește și numărul de conexiuni: fiecare componentă va avea O(N) conexiuni, deci numărul total va fi:

$$\frac{n(n-1)}{2}$$

Treptat, un asemenea sistem ajunge să "scape de sub control". Adăugarea de noi funcționalități - care era foarte simplă în stadiile inițiale - devine ineficientă din cauza overheadului de conectare.

Integrarea verticală

Integrarea verticală presupune crearea de componente cvasi-autonome. În acest scop, fiecare componentă va duplica părți din funcționalitatea celorlalte, inclusiv storage.

Avantajul este că, odată ce componentele au fost concepute și implementate în acest fel, ele pot fi integrate în sistem cu costuri și impact minim. Dezavantajul constă în costul mai ridicat al componentelor, care nu refolosesc funcționalități și module, ci mai curând le duplică.

Integrarea orizontală

Integrarea orizontală - sau Enterprise Application Integration [http://en.wikipedia.org/wiki/Enterprise_application_integration] (EAI) - presupune crearea unei componente specializate (*middleware [http://en.wikipedia.org/wiki/Middleware]*) care este dedicată comunicației între celelalte componente. Această soluție permite reducerea numărului de conexiuni (interfețe) la una singură pentru fiecare subsistem, urmând ca subsistemul middleware să asigure translatarea unei interfețe în alta (respectiv să "ruteze" datele de la o componentă la alta).

Avantajele sunt multiple: pe de o parte componentele au nevoie de o singură interfață, pe de altă parte ele pot refolosi funcționalitățile celorlalte. Componentele rămân separate, putând fi înlocuite sau dezvoltate fără afectarea celorlalte.

Patternuri de integrare orizontală

Din punct de vedere funcțional, într-o integrare de tip orizontal apar doua patternuri tipice, de obicei complementare:

- **Intermediere**: intermedierea conectivității componentelor. Ori de câte ori are loc un eveniment (sau o cerere) într-una din componente, aceasta este distribuită corespunzător către celelalte componente prin interfețele acestora. Componentele folosesc structuri proprii pentru mesajele cu care comunică; mesajele sunt transformate din forma expeditorului într-o structură specifică destinatarului.
- **"Federation"**: legarea sistemului cu mediul exterior prin intermediul interfețelor publice și ascunzând interfețele private. Componentele cad de acord asupra structurii mesajelor și toate folosesc aceiași structură.

Din punct de vedere al propagării evenimentelor sau cererilor, apar de asemenea două situații tipice de transmitere a mesajelor:

- **Sincronă**: Este acea comunicare în care toate părțile participă în același timp la curgerea mesajelor. Aceasta necesită transmiterea mesajului de un participant X, blocarea lui până la primirea răspunsului, prelucrarea mesajului de un participant Y, trimiterea răspunsului înapoi la X. Un exemplu este convorbirea la telefon, în care e necesar ca cel cu care vreau să vorbesc să fie lângă telefon în momentul în care sun. Avantajul acestui tip de comunicare îl reprezintă simplitatea, dar apar dezavantaje precum pierderea timpului, blocarea unui thread în asteptarea răspunsurilor, suprasolicitarea unui destinatar comun.
- **Asincronă**: În această abordare transmițătorul X doar trebuie să aibă grijă că mesajul ajunge în canalul de transmisie; canalul având grijă ca mesajul să ajungă cu bine. Destinatarul, sau receptorul are o coadă în care primește mesajele și le prelucrează la viteza maximă de care dispune. În acest timp expeditorul face alte activități, nefiind blocat în așteptarea unui răspuns. Exemplu ar fi lăsarea unui mesaj în căsuța vocală a celui cu care dorim să comunicăm. Avantajele sunt evidente: separarea funcțiilor, randament maxim, evitarea suprasolicitării; dar toată arhitectura trebuie gândită într-un mod mai complex bazat pe evenimente.

Din punct de vedere al tipurilor de date transmise, acestea pot fi transferate ca și:

- **Fișiere**: O aplicație scrie un fișier pe care o alta îl citește mai târziu. Aplicațiile trebuie să cadă de acord asupra numelui de fișier și locației, formatului de fișier, momentului când acesta va fi scris și citit, și cine va șterge fișierul.
- **Bază de date comună**: Aplicații multiple împărtășesc aceeași schemă de baze de date, situată într-o singură bază de date fizică. Deoarece nu există duplicate ale bazei de date, datele nu trebuie să fie transferate la o aplicație la alta.
- **Apelare de proceduri la distanță**: O aplicație expune anumite funcționalități astfel încât acestea să poată fi accesate de la distanță de alte aplicații, ca și proceduri la distanță. Comunicarea are loc în timp real și sincron.
- **Mesaje**: Aplicatiile trimit mesaje printr-un canal de mesaje comun. Alte aplicații pot citi mesajele de pe canal la un moment ulterior. Aplicațiile trebuie să cadă de acord asupra unui canal, precum și formatului mesajelor. Comunicarea este asincronă.

Service Oriented Architecture (SOA)

SOA [http://en.wikipedia.org/wiki/Service-oriented_architecture] reprezintă un set de guidelines de arhitectură pentru sisteme **scalabile** și **extensibile**. Ideea de bază este ca fiecare componentă software să își exporte funcționalitățile sub forma unui **serviciu** accesibil remote, ceea ce deschide calea sistemelor complexe, flexibile și refolosibile. SOA nu este propriu zis o rețetă, ci o colecție de principii:

1. **Incapsulare** - componentele nu au restricții de arhitectură internă
2. **Cuplare slabă** - componentele trebuie să aibă dependințe reciproce cât mai mici
3. **"Contract de servicii"** - componentele aderă la o specificație comună de comunicare și conlucrare reciprocă
4. **Abstractizare** - componentele nu expun public logica lor internă
5. **Refolosire** - împărțirea pe servicii este gândită în vederea refolosirii
6. **Combinabilitatea** - serviciile pot fi combinate și încapsulate sub forma unui serviciu
7. **Autonomie** - serviciile au control total asupra logicii lor interne
8. **Optimizare** - între două servicii echivalente, se va prefera serviciul mai bun din punct de vedere calitativ
9. **Descoperire** - serviciile exportă informații care permit descoperirea și folosirea lor de către alte servicii
10. **Relevanță** - serviciile trebuie să aibă valoare pentru end-useri

Exemplu: Flickr.com, Google.com, Facebook.com - toate sunt gândite ca SOA, exportând servicii prin intermediul unui API. Consecințe: funcționalități ca Google search sau Google maps sunt integrabile în

contexte (pagini sau aplicații Desktop) în afara lui Google. În mediul web, acest gen de servicii integrabile în cvasi-orice context se numesc widgets [http://www.widgipedia.com].

Platforme pentru integrare (5 min)

Există multe soluții software [http://en.wikipedia.org/wiki/Comparison_of_business_integration_software] care încearcă să automatizeze integrarea diverselor componente software. Cele mai cunoscute sunt următoarele două:

Application servers

Un application server [http://en.wikipedia.org/wiki/Application_server] este un framework care integrează componentele în jurul unui web server. Componentele comunică prin intermediul serverului. Exemple:

- Apache Geronimo (Apache)
- JBoss (RedHat)
- WebSphere Application Server (IBM)
- WebLogic Server (Oracle)

Enterprise Service Bus

Un enterprise service bus [http://en.wikipedia.org/wiki/Enterprise_Service_Bus] este un framework având la bază un layer generic de comunicare între diferite componente software. Comunicarea se face prin intermediul *mesajelor*. Exemple:

- WebSphere Enterprise Service Bus (IBM)
- Enterprise Service Bus (Oracle)
- Progress Sonic ESB

Integration testing (5 min)

Testarea integrării componentelor [http://en.wikipedia.org/wiki/Integration_testing] se face pentru verificarea funcționalității, performanței și stabilității sistemelor compuse. Este un sistem tip cutie neagră, în sensul că testarea nu se face la nivelul componentelor sistemului, ci la nivelul interfețelor acestora. Cazurile de succes și eroare sunt testate prin inputuri (parametri și date) simulate corespunzător.

Scenariile de testare urmăresc să verifice felul în care componentele interacționeaza. Această etapă de testare este *ulterioară* testării individuale a componentelor (i.e. unit testing) și presupune că toate componentele sunt deja funcționale și corespund specificațiilor.

Exista trei abordări [http://msdn.microsoft.com/en-us/library/aa292128] principiale pentru testarea integrării:

Botom-up

Testarea bottom-up presupune testarea componentelor de la nivelul cel mai jos, urcând progresiv către componentele de nivel înalt până către ansamblul sistemului. La nivelul cel mai de jos, se integrează procedurile sau funcțiile în modulele respective, apoi sunt testate. După aceea, se integrează modulele și se face o nouă rundă de testare. Acest tip de testare este foarte eficace în găsirea bugurilor, și de asemenea ușor de monitorizat (se poate ști în fiecare moment ce procent din teste a fost terminat). Are însa marele dezavantaj că nu testează logica și flow-urile principale de date până foarte târziu în cadrul proiectului, ceea ce face ca eventualele greșeli de design să fie depistate relativ târziu. De asemenea, nu permite un eventual release parțial, cu funcționalități limitate.

Top-down

Testarea top-down presupune integrarea și testarea modulelor începând cu cele de la nivelul cel mai inalt. Avantajul este că logica și flowurile principale de date sunt testate mai devreme, deci orice greșeala de design este depistată în timp util. Dezavantajul constă însă în overheadul mare necesar pentru test cases și pentru simulările de date, în condițiile în care sistemele nu sunt terminate și funcționale. Alt dezavantaj este

că modulele de nivel jos sunt testate relativ târziu în cadrul proiectului. Nici acest model nu permite un release parțial, cu funcționalități limitate.

"Umbrella" sau "Sandwich"

Este o combinație a celor două metode de mai sus. În prima fază, se testează funcțiile de nivel jos, ca în cazul abordării bottom-up. Ieșirile funcțiilor sunt apoi integrate și testate în maniera top-down. Este o metodă mai puțin sistematică decât oricare dintre cele două anterioare, și mai puțin exactă, însă avantajul este că permite release-uri parțiale, cu funcționalități limitate, înainte de terminarea întregului proiect.

Exerciții

Story telling (30 de minute)

Formați echipe de 2-3 persoane. O echipă pornește cu o poveste formată din 3-4 fraze. Celelalte echipe vor construi **simultan** a 2-a, a 3-a, a 4-a parte a poveștii. Vedeți la sfârșit ce a ieșit.

Exercitiu integrare (40 de minute)

- Identificati componentele proiectului. Ce abordare de integrare considerati ca este potrivita pentru proiectul vostru? Justificati alegerea.
- Alegeti abordarea de integration testing potrivita pentru proiectul vostru. Justificati alegerea.
- Prezentati alegerile facute in fata colegilor. Aduceti argumente pro pentru alegerile voastre / contra alegerilor colegilor.

Bibliografie

- http://www.enterpriseintegrationpatterns.com/Introduction.html [http://www.enterpriseintegrationpatterns.com/Introduction.html]
- http://msdn.microsoft.com/en-us/library/ms978729.aspx [http://msdn.microsoft.com/en-us/library/ms978729.aspx]

mps/laboratoare/laborator-08.txt · Last modified: 2020/12/02 15:02 by iulia.stanica